

Survey of the Existing Passes

Compiler Engineering #4: Finding, reading, and reusing the LLVM middle end

Gen Sakai

2026/08/01

B4, Taura Laboratory, The University of Tokyo

Contents

- 1. Session Bridge 2
- 2. Finding What You Do Not Know 5
- 3. The New Pass Manager CLI 11
- 4. Helper Passes 15
- 5. Canonicalization 21
- 6. Optimization Passes 29
- 7. Hands-On 42

1. Session Bridge

Sessions 1–3: how to **build** LLVM, what IR **is**, and how to **write** a pass – CMake and tooling, the **Module / Function / BasicBlock / Instruction** graph, SSA and dominance, pass scopes and pass managers.

So we can now write a pass. That is exactly the moment the textbook interrupts:

*“There is zero benefit in reimplementing your own **mem2reg** pass; at best your implementation will be equivalent and at worst you’ll introduce a bunch of bugs!”*

Today: the middle end as a **catalog**. Not “how do I write a pass” but “what already exists, how do I find it, and how do I read it”.

Theme	What we focus on
Chapter 8	Finding unknown passes; helper, canonicalization, and optimization passes

Chapter 8 sits at a hinge in the book:

- **Ch. 4/5 (sessions 2–3)** gave us the **machinery**: SSA, dominance, passes, pass managers
- **Ch. 8 (today)** gives us the **inventory**: what LLVM already ships, and how to discover the rest ourselves

The chapter has only two stated goals: make you aware of what the middle end can do, and teach you how to approach a transformation you do not know. The second one is the real subject.

2. Finding What You Do Not Know

The Problem

If someone names a pass, `git grep` finds it. But:

*“if you do not know what exists in the middle end,
how do you even begin to look for it?”*

Two directions, which the chapter’s summary names explicitly:

Direction	Route
Top-down	<code>opt</code> ’s help message → the implementation
Bottom-up	directory structure and filenames → implementation → back to <code>opt</code> ’s CLI

Both routes converge on the same place: a source file. The last slide of this section is what to do once you are there.

Top-Down: opt as the Index

opt drives **every** LLVM-IR-to-LLVM-IR pass, so its help message is an index of the middle end.

```
$ opt --help          # all passes, both pass managers, with
                        descriptions
$ opt --print-passes  # only what the NEW pass manager supports, by
                        scope
```

--print-passes groups by scope, which is the useful part:

Module passes:	Function passes:	Machine function passes (WIP):
always-inline	adce	machine-cp
attributor	dce	machine-scheduler
CGSCC passes:	Loop passes:	Machine function analyses (WIP):
inline	indvars	machine-dom-tree
argpromotion	licm	machine-loops

Recipe: take **--print-passes**, drop the **Machine ...** sections → that is the middle end. Intersect with **--help** → you now have a name **and** a description for each.

opt Is Lying To You

`--help` mixes both pass managers indiscriminately, and the legacy one also drives backends. So some listed passes are not IR-to-IR passes at all, and `opt` cannot run them:

```
--aa      - Function Alias Analysis Results
--aarch64-00-prelegalizer-combiner - Combine AArch64 machine
instrs ...
--gvn     - Global Value Numbering
```

- `--passes=gvn` works
- `--passes=aarch64-00-prelegalizer-combiner` fails – fine, the name says `aarch64`
- `--passes=aa` **also fails** – and `aa` is target-independent. **The name does not tell you.**

That is why `--print-passes` is a mandatory second command, not a nicety.

A target-specific pass you cannot run may still be worth reading: generalize it for your case, or use it as inspiration for your own pass.

Bottom-Up: The Directory Map

Generic IR passes live in exactly two trees (`include/llvm/` and `lib/`): **Analysis/** and **Transforms/**. **Transforms/** subdivides:

Subdirectory	Contents
InstCombine	simple rewrite patterns beneficial for all targets
AggressiveInstCombine	a more aggressive version of the above
IP0	interprocedural – applies across functions (e.g. inlining)
Scalar	everything non-vector. Most passes are here , loops included
Vectorize	produces vector-typed IR (SIMD)
Utils	generally useful passes and helper functions
Instrumentation	adds bookkeeping next to the IR: PGO, sanitizers
Coroutines	coroutine lowering – highly source-language specific
HipStdPar	HIP C++ standard parallelism support

*This table alone answers quiz 3. “Where is CSE?” It is an optimization (**Transforms**), function-scoped (not **IP0**), non-vector (not **Vectorize**) → **Scalar** . And indeed:
`llvm/lib/Transforms/Scalar/EarlyCSE.cpp` .*

Landing on the Implementation

Both routes end at a file. From a CLI name to that file:

```
# via the registration macro you met in Chapter 5
$ git grep "always-inline" llvm/lib/Passes/PassRegistry.def
MODULE_PASS("always-inline", AlwaysInlinerPass())
$ git grep -l "class AlwaysInlinerPass" llvm
llvm/include/llvm/Transforms/IPO/AlwaysInliner.h

# or via the description string from --help
$ git grep -l "Inliner for always_inline functions" llvm
llvm/lib/Transforms/IPO/AlwaysInliner.cpp
```

Then, in the file: **read the header comment** (often detailed; sometimes cites the paper the pass is based on), and find its tests:

```
$ git grep -l 'RUN: .*always-inline' llvm/test
```

*The tests are the real documentation. Extract the **RUN:** command, then **edit the input IR and re-run** – that is how you find out what a pass actually does, as opposed to what its comment claims.*

3. The New Pass Manager CLI

Scope Is In The Syntax

Session 3 left the new pass manager as an IOU. Chapter 8 needs exactly one thing from it: **the pipeline string encodes scope**.

```
$ opt --passes='function(print,consthoist,loop(print),  
                    instcombine<use-loop-info;max-  
iterations=3>),globaldce'
```

- `function(...)`, `loop(...)`, `module(...)` open a nested pipeline – these are the `XXXPassAdaptor`s from the Chapter 5 exercise
- with no keyword, **the scope is inferred from the first pass you name**
- `print` is registered once for all three scopes; which one runs depends on the enclosing scope (`PrintModulePass` / `PrintFunctionPass` / `PrintLoopPass`)
- `<...>` passes pass-specific flags – **implementation-specific and not stable across releases**

Docs: <https://llvm.org/docs/NewPassManager.html#invoking-opt>

A Scoping Trap (Try This)

Quiz 4 asks: after `indvars`, which pass removes the leftover constant branch?

Answer: `simplifycfg`. So chain them:

```
$ opt -passes='indvars,simplifycfg' indvars.ll -S
opt: unknown loop pass 'simplifycfg'
```

`indvars` is a **Loop** pass, so the inferred scope is loop – and `simplifycfg` is a **Function** pass. Name the scopes and it works:

```
$ opt -passes='loop(indvars),simplifycfg' indvars.ll -S
```

```
$ opt -passes='dce,globaldce' dce.ll -S
opt: unknown function pass 'globaldce'      # same trap, function
vs module
```

*The error says “unknown **loop** pass”, not “unknown pass” – it is telling you the inferred scope. Reading that one word saves a lot of confusion.*

A Scope the Book Does Not Mention

There are **two** loop scopes, and picking the obvious one breaks **licm**:

```
$ opt -passes='licm'          licm.ll -S      # fine
$ opt -passes='loop(licm)'    licm.ll -S
LLVM ERROR: LICM requires MemorySSA (loop-mssa)
$ opt -passes='loop-mssa(licm)' licm.ll -S    # fine
```

Scope	Guarantees it establishes
loop(...)	LCSSA form, innermost-first order
loop-mssa(...)	the same, plus a maintained MemorySSA

licm's whole legality argument is about memory, so it needs **MemorySSA**. Two traps in one:

- **being explicit is what breaks it** – the inferred scope was already **loop-mssa**, and you overrode a correct default
- **it fails at run time, not parse time** – **opt** accepts the string and only aborts when a loop reaches **licm**. On input with no loops, it passes silently

*The scope keyword is not decoration. It selects which guarantees the pass manager establishes before your pass runs – which is also why entering **loop(...)** materialised LCSSA form on the printer slide.*

4. Helper Passes

The Verifier

Optional, never modifies the IR – and **critical for every compiler writer**. It checks the IR is well formed: that operand types agree, and that

every definition dominates all of its uses – i.e. the IR really is in SSA form.

```
$ opt -passes=verify use_before_def.ll
Instruction does not dominate all uses!
  %a = add i64 1, 2
  %res = add i64 %a, 3
error: input module is broken!
```

- **llc** runs it by default across the codegen pipeline (**-disable-verify** to stop)
- **opt -verify-each** inserts it **after every pass** in your pipeline
- in the NPM: **VerifierPass(bool FatalErrors)**, or **VerifyEachPass=true** on **StandardInstrumentations**

*Quiz 2: an existing pass crashes on your custom pass's output. Before debugging **their** pass, run the verifier on **your** output. Invalid IR must never be fed to a pass – passes may then behave in arbitrary ways.*

The Printers

One printer per scope, so you can look at the IR at any point in a pipeline.

```
$ opt -print-after-all          # IR after every pass
$ opt -print-before=instcombine # IR before one named pass
$ opt --passes='function(print,loop(print))' # positional, nested
```

Run that last one and compare the two copies of `%end` it prints:

In

```
; PrintFunctionPass
end:
  %tmp = add i64 %iv_plus_1, %src
  %res = add i64 %tmp, %iv_plus_1
  ret i64 %res
```

Out

```
; PrintLoopPass (+ Preheader/Loop/Exit labels)
end:
  %iv_plus_1.lcssa = phi i64 [%iv_plus_1,%loop]
  %tmp = add i64 %iv_plus_1.lcssa, %src
  %res = add i64 %tmp, %iv_plus_1.lcssa
```

We never asked for `lcssa`. **Entering the loop scope materialised LCSSA form**, because

`LoopAnalysisManager` guarantees loop passes receive loops that way. The scope keyword did not only pick which printer runs – it changed the IR the printer saw.

Same registered name `print`, three different passes. And a free demonstration of a guarantee we only stated in words two slides from now.

Analysis Passes

Analysis	NPM class	Result	CLI
Target transform info	TargetIRAnalysis	TargetTransformInfo	target-ir
Loop info	LoopAnalysis	LoopInfo	loops
Alias analysis	AAManager	AAResults	aa
Block frequency	BlockFrequencyAnalysis	BlockFrequencyInfo	block-freq
Dominator tree	DominatorTreeAnalysis	DominatorTree	domtree

```
// legacy PM
auto &TTI =
getAnalysis<TargetTransformInfoWrapperPass>().getTTI(F);
// new PM
auto &TTI = FAM.getResult<TargetIRAnalysis>(F);
```

- most analyses have **no** observable effect: `require<target-ir>` computes it, but if nothing consumes it you see nothing. The lucky ones support `print<loops>`, `print<domtree>`, `print<block-freq>`
- `domtree` lives in the **IR** library, not **Analysis** – it is inseparable from SSA. Use `DomTreeUpdater` if you maintain it yourself

Alias Analysis Is Not One Analysis

It exposes one question – “can these two pointers alias?” – but answers it by **consolidating several analyses that augment each other**.

The textbook’s example of two of them:

Analysis	Reasoning	Cost
type-based	under strict aliasing, a <code>float*</code> cannot alias an <code>int*</code>	cheap
range-based	conservatively bound the memory a pointer can reach	expensive

If the cheap one settles it, the expensive one never runs.

- drive it through `AAManager` ; add more with `AAManager::registerFunctionAnalysis`
- `PassBuilder::buildDefaultAAPipeline` already wires the common **and** target-specific ones

This is the analysis behind session 3’s legality example – whether `B[1] = val2` invalidates a cached `A[0]`. It is also why `licm` needs to prove nothing in the loop writes memory before hoisting a load.

Value Tracking

Not a pass – a helper class in the `Analysis` library. Give it a value, get back the **known bits**.

```
%a      = and i64 %b, u0xffffffffffffc
%mod     = urem i64 %a, 2
%cond    = icmp eq i64 %mod, 0
```

We do not know `%b`. But the mask forces the **low two bits** of `%a` to zero, so `%a` is a multiple of 4, so `%mod` is always `0`, so `%cond` is always `true`:

```
$ opt -passes=instcombine value_tracking.ll -S
define i1 @foo(i64 %b) { ret i1 true }
```

```
KnownBits computeKnownBits(const Value *V, const DataLayout &DL, /
*...*/);
```

*The book says “the first three least significant bits”; the published errata corrects this to **two**, which is what the mask actually does. Easy to use, very powerful – but it can be compile-time intensive.*

5. Canonicalization

What Canonical Means

Session 1 introduced the term (`a = b + 2` vs `a = 2 + b`). Now the consequences. For `a = b - c` a frontend may emit either:

In

```
; canonical  
%a = sub i64 %b, %c
```

Out

```
; equally valid, not canonical  
%neg_c = sub i64 0, %c  
%a = add i64 %b, %neg_c
```

Same semantics. LLVM's canonical form is the left one. Two consequences:

- in the standard pipeline, **anything not canonical will be canonicalized**
- **optimizations are tested almost exclusively on the canonical form** – feed them non-canonical IR and they miss opportunities or hit rough edges

Then why not canonicalize on construction? Because one target's preferred form is another's adverse form – the right-hand version suits a target with add and negate but no subtract. LLVM deliberately leaves the choice representable.

instcombine: Canonicalize

InstCombinePass (InstructionCombiningPass)
--

instcombine

`ch8/canonical_form.ll` holds both spellings of `b - c`. One pass, and they converge:

In

```
define i64 @canonical_form(  
    i64 %b, i64 %c) {  
    %a = sub i64 %b, %c  
    ret i64 %a  
}  
  
define i64 @non_canonical_form(  
    i64 %b, i64 %c) {  
    %neg_c = sub i64 0, %c  
    %a = add i64 %b, %neg_c  
    ret i64 %a  
}
```

Out

```
define i64 @canonical_form(  
    i64 %b, i64 %c) {  
    %a = sub i64 %b, %c  
    ret i64 %a  
}  
  
define i64 @non_canonical_form(  
    i64 %b, i64 %c) {  
    %a = sub i64 %b, %c      ; <-- rewritten  
    ret i64 %a  
}
```

A canonical rewrite need not be *faster*. The `inttoptr` example in the book expands an implicit $64 \rightarrow \{32\}$ -bit truncation into an explicit `trunc` – same cost, but now *other* passes can see and simplify the added logic.

The second half of what `instcombine` does is generically-useful optimization:

In

```
define i64 @xor(i64 %x) {  
  %res = xor i64 %x, %x  
  ret i64 %res  
}
```

Out

```
define i64 @xor(i64 %x) {  
  ret i64 0  
}
```

This is why `instcombine` is **not** in the `-O0` pipeline. And that has a consequence worth stating out loud:

*at `-O0` the IR is mostly **not** in canonical form*

`-O0` is garbage-in-garbage-out by design. But your **lowering** passes run at every optimization level – so **they must be correct on non-canonical IR**.

Scale check before you go reading: `llvm/test/Transforms/InstCombine` is roughly 1,500 files and 32k IR function definitions. The book's advice is to use it, not to read it.

instcombine: Pipeline Discipline

Most passes never worry about canonical form – they rely on `instcombine` being **re-run periodically to clean up**. Building your own pipeline, that becomes your job:

- insert it in **several** places – but it is a **compile-time sink** if run too often
- once you introduce target-specific constructs, `instcombine` may **undo them** by restoring the canonical form (`add %b, (sub 0, %c)` → `sub %b, %c`)
- so the typical shape is: `instcombine` heavily **until** target-specific passes appear, then never again

Still want the simplifications after that point?

```
// Analysis library -- the simplification logic without the pass
Value *simplifyAddInst(Value *LHS, Value *RHS, bool IsNSW, bool
IsNUW, ...);
Value *simplifyBinOp(unsigned Opcode, Value *LHS, Value *RHS, ...);
```

The `simplifyXXXInst` helpers give you `instcombine`-grade folds without the canonicalization that would revert your target-specific work.

PromotePass (PromoteLegacyPass)

mem2reg

Replaces memory traffic with SSA values – the pass session 2 kept referring to.

In

```
define i64 @foo(i64 %in) {  
entry:  
  %slot = alloca i64  
  store i64 %in, ptr %slot  
  %v = load i64, ptr %slot  
  %res = add i64 %v, 2  
  ret i64 %res  
}
```

Out

```
define i64 @foo(i64 %in) {  
entry:  
  %res = add i64 %in, 2  
  ret i64 %res  
}
```

Not really a canonicalization, but grouped here because **its output is the starting point of any sane optimization**. Without it every pass would have to track memory locations and use alias analysis to do anything – and you would get **none** of SSA's def-use chains or dominance.

The book's phrasing: you would be “stuck with a compiler technology of another age”. Run it as early as possible – but not at -O0.

LCSSAPass (LCSSAWrapperPass)

lcssa

Loop-closed SSA guarantees **no value defined in a loop is used outside it**, by adding phis in the exit blocks.

In

```
loop:
  %iv = phi i64 [0,%entry],[%iv_plus_1,%loop]
  %iv_plus_1 = add i64 %iv, 1
  %cond = icmp ult i64 %iv_plus_1, %ub
  br i1 %cond, label %loop, label %end

end:
  %tmp = add i64 %iv_plus_1, %src
  %res = add i64 %tmp, %iv_plus_1
  ret i64 %res
```

Out

```
loop:
  %iv = phi i64 [0,%entry],[%iv_plus_1,%loop]
  %iv_plus_1 = add i64 %iv, 1
  %cond = icmp ult i64 %iv_plus_1, %ub
  br i1 %cond, label %loop, label %end

end:
  %iv_plus_1.lcssa = phi i64 [%iv_plus_1,%loop]
  %tmp = add i64 %iv_plus_1.lcssa, %src
  %res = add i64 %tmp, %iv_plus_1.lcssa
  ret i64 %res
```

Now walking `%iv_plus_1`'s def-use chain gives you **only in-loop uses** (plus the `.lcssa` phi itself). Uses outside the loop moved to a separate value.

*A GPU angle from the book: inside the loop the induction variable is identical across lockstep threads, so it can live in a cheap **scalar** register – but not outside, since `%ub` may differ per thread. The `.lcssa` value captures exactly that boundary.*

LCSSA Is Ephemeral

Add `instcombine` after `lcssa` and the phi is gone again:

```
$ opt -passes='lcssa,instcombine' lcssa.ll -S  
end:  
    %reass.add = shl i64 %iv_plus_1, 1  
    %res = add i64 %src, %reass.add
```

No `.lcssa` value survives. So:

- put `lcssa` **immediately in front** of the pass that needs it, not “somewhere earlier”
- if you write a **loop pass**, you get it for free: `LoopAnalysisManager` guarantees the loops handed to your `run` method are already in LCSSA form

That guarantee is why `licm`'s output two sections from now contains an `.lcssa` phi that nobody asked for. Every loop pass sees loops in this form.

6. Optimization Passes

The Identity Card

Unlike canonicalization passes, these exist purely to make code faster – and **may be irrelevant to your target**. The book gives each one a card:

ClassName (LegacyName, or NA)	cli-name
-------------------------------	----------

Description	what it does, at a high level
In / Out	an example transformation
Explanation	what happened between the two
Target-specific	which target APIs steer it, if any

Every input lives in `ch8/<cli_name>.ll`, so each card is reproducible in one command. **NA** in the legacy slot means **no legacy equivalent exists** – a visible marker of the ongoing NPM migration from session 3.

The book explicitly invites you to extend the list or invent your own card format. Exercise 3 of this module does exactly that.

IPO: argpromotion

ArgumentPromotionPass (NA)	argpromotion
----------------------------	--------------

Turns a by-reference argument into a by-value one.

In

```
define i64 @foo() {  
    %local = alloca i64  
    store i64 2, ptr %local  
    %res = call i64 @bar(ptr %local)  
    ret i64 %res  
}  
  
define internal i64 @bar(ptr %local) {  
    %val = load i64, ptr %local  
    %res = add i64 %val, 2  
    ret i64 %res  
}
```

Out

```
define i64 @foo() {  
    %local = alloca i64  
    store i64 2, ptr %local  
    %local.val = load i64, ptr %local  
    %res = call i64 @bar(i64 %local.val)  
    ret i64 %res  
}  
  
define internal i64 @bar(i64 %local.0.val) {  
    %res = add i64 %local.0.val, 2  
    ret i64 %res  
}
```

Only legal because `@bar` is `internal`: the pass sees **all** uses, so it may change the ABI. The load did not disappear – it **moved into the caller**. That is the point: now `mem2reg` can delete the `alloca`, `store`, and `load` outright.

Target-specific: `TargetTransformInfo`, to check the new signature is compatible with the target ABI.

IPO: deadargelim and inline

DeadArgumentEliminationPass (DAE)

deadargelim

Drops unused parameters from a signature – again only when all call sites are visible. Removing `@bar`'s unused second argument leaves `%local2`'s `alloca / store` dead in the caller. **Target-specific: none.**

InlinerPass (NA)

inline

Replaces a call with the callee's body. Two things worth naming:

- it uses **CGSCC** regions to order decisions, from the leaves of the call graph upward – session 3's "children-to-parents traversal", now with a purpose. Want a different order? **module-inliner**
- in the book's example `@bar` **survives** inlining: its linkage is not **internal**, so another module might still call it

Target-specific: **InlineAdvisorAnalysis** makes each decision, and by default derives a cost from

TargetTransformInfo and compares it to a threshold. This is session 3's "no silver bullet for profitability" as an actual, replaceable component.

DCEPass (DCELegacyPass)

dce

If a value has no users and its instruction has no side effects, it goes.

In

```
define i64 @foo(i64 %in) {  
    %dead = add i64 %in, %in  
    %res = mul i64 %in, 2  
    ret i64 %res  
}
```

Out

```
define i64 @foo(i64 %in) {  
    %res = mul i64 %in, 2  
    ret i64 %res  
}
```

You already wrote a miniature of this pass: in session 2's `ir-ssa-lab`, `isInstructionTriviallyDead` plus `eraseFromParent` after `replaceAllUsesWith` left the folded `add`/`mul` with zero users.

Target-specific: `TargetLibraryInfo`, to decide whether values produced by library calls and intrinsics are safe to remove – the part your lab pass did not have to handle.

IndVarSimplifyPass (NA)

indvars

Rewrites values derived from induction variables into a form other passes can use.

In

```
define i64 @foo(i64 %src, i64 %ub) {
entry:
  br label %loop
loop:
  %iv = phi i64 [0,%entry],[%iv1,%loop]
  %iv1 = add i64 %iv, 1
  %cond = icmp ult i64 %iv1, %ub
  br i1 %cond, label %loop, label %end
end:
  %tmp = add i64 %iv1, %src
  %res = add i64 %tmp, %iv1
  ret i64 %res
}
```

Out

```
define i64 @foo(i64 %src, i64 %ub) {
entry:
  br label %loop
loop:
  br i1 false, label %loop, label %end

end:
  %umax = call i64 @llvm.umax.i64(
    i64 %ub, i64 1)
  %tmp = add i64 %umax, %src
  %res = add i64 %tmp, %umax
  ret i64 %res
}
```

The trip count is statically known, so the whole `%iv / %iv1` chain is pushed out of the loop and re-expressed as a function of `%ub`. The body collapses to a constant-condition branch – **left for another pass to clean up**.

Target-specific: `TargetLibraryInfo` (did library calls become side-effect-free?) and `TargetTransformInfo` (is the new sequence cheaper?).

LICMPass (LegacyLICMPass)

licm

Hoists computations that do not change across iterations out of the loop.

In

```
entry:
  br label %loop

loop:
  %iv = phi i64 [0,%entry],[%iv1,%loop]
  %offset = load i64, ptr %addr
  %iv1 = add i64 %iv, %offset
  %cond = icmp ult i64 %iv1, %ub
  br i1 %cond, label %loop, label %end

end:
  %res = add i64 %src, %iv1
```

Out

```
entry:
  %offset = load i64, ptr %addr
  br label %loop

loop:
  %iv = phi i64 [0,%entry],[%iv1,%loop]
  %iv1 = add i64 %iv, %offset
  %cond = icmp ult i64 %iv1, %ub
  br i1 %cond, label %loop, label %end

end:
  %iv1.lcssa = phi i64 [%iv1,%loop]
  %res = add i64 %src, %iv1.lcssa
```

Legality is the whole story: hoisting the load is safe only because **nothing in the loop writes memory**, so no aliasing can invalidate it – session 3's **A[0]** / **B[1]** example, decided by alias analysis. And note the **.lcssa** phi appearing unbidden: **licm** is a loop pass.

Target-specific: *TargetLibraryInfo* for legality, *TargetTransformInfo* for profitability.

Scalar: loop-reduce

LoopStrengthReducePass (LoopStrengthReduce)

loop-reduce

Strength reduction = replace a computation with a cheaper equivalent ($x * 2 \rightarrow x \ll 1$). This pass does it for **address** computations built from induction variables, to fit the target's **addressing modes**.

In

```
%i5 = getelementptr inbounds i64,  
    ptr %arg, i64 %idx  
%i6 = load i64, ptr %i5
```

Out

```
%0 = shl i64 %idx, 3  
%scevgep = getelementptr i8,  
    ptr %arg, i64 %0  
%i6 = load i64, ptr %scevgep
```

$A[i]$ in C is $*(A + i * \text{sizeof}(A[0]))$, so lowering introduces a multiply – here $\%idx * 8$ from the **i64** element type. The rewrite applies the scale with a cheaper **shl**, then switches the GEP to **i8**, whose scaling factor is 1 and disappears. Many targets fold exactly this shape into an addressing mode (the multiplier is the **scaling factor**).

Target-specific: **TargetTransformInfo** – and note **loop-reduce.ll** declares a target triple.

TargetTransformInfo needs a real **TargetLowering** underneath to answer which addressing modes exist. The pass also splits critical edges and applies LCSSA to simplify its own job.

Scalar: loop-unroll and reassociate

LoopUnrollPass (LoopUnroll)	loop-unroll
-----------------------------	-------------

Expands the body so one new iteration does N old ones; a constant trip count allows **full** unrolling into straight-line code. Goals: fewer branches, and **new opportunities for vectorization and scheduling**.

Target-specific: `TargetTransformInfo` for the cost, and targets may override `TargetTransformInfo::getUnrollingPreferences` to steer the thresholds.

ReassociatePass (ReassociateLegacyPass)	reassociate
---	-------------

In

```
%v0 = add i64 %in0, %in1
%v1 = add i64 %v0, 2
%v2 = sub i64 %v1, %in1
ret i64 %v2
```

Out

```
%v2 = add i64 %in0, 2
ret i64 %v2
```

`in0 + in1 + 2 - in1` reordered to `in0 + in1 - in1 + 2`, then folded. It groups constants for compile-time evaluation, pairs up cancelling terms, and exposes CSE. **Target-specific: none.**

Reassociation's hard part is **legality**: reordering floating-point arithmetic accumulates rounding differently, so the result can change. That is what session 3's fast-math flags are for.

SimplifyCFGPass (CFGSimplifyPass)

simplifycfg

Removes useless control flow – notably branches with a constant condition. Feed it the **output of indvars** from three slides ago:

In

```
entry:
  br label %loop

loop:
  br i1 false, label %loop, label %end

end:
  %umax = call i64 @llvm.umax.i64(i64 %ub, i64 1)
  %tmp = add i64 %umax, %src
  %res = add i64 %tmp, %umax
  ret i64 %res
```

Out

```
entry:
  %umax = call i64 @llvm.umax.i64(i64 %ub, i64 1)
  %tmp = add i64 %umax, %src
  %res = add i64 %tmp, %umax
  ret i64 %res
```

Step by step: **br i1 false** always goes to **end**, so it becomes unconditional and the blocks merge; **entry** then has a single successor with a single predecessor, so **those** merge too. Four blocks → one.

Target-specific: *TargetTransformInfo* for its cost model. Also available as the *simplifyCFG* API in *TransformUtils*, without the pass.

Vectorization

Scalar $\rightarrow$ SIMD: two unrelated adds $a = b + c$ and $d = e + f$ become one $\langle a, d \rangle = \langle b, e \rangle + \langle c, f \rangle$, each lane in parallel. Three passes, three angles of attack:

Pass	What it looks for
load-store-vectorizer	the longest chain of same-kind accesses to contiguous addresses
slp-vectorizer	a straight-line sequence of similar scalar instructions
loop-vectorize	a loop body that can process several iterations at once

In

```
%v0 = add i64 %in0, 2
%v1 = add i64 %in1, 5
%partial = insertelement
    <2 x i64> poison, i64 %v0, i32 0
%res = insertelement
    <2 x i64> %partial, i64 %v1, i32 1
```

Out

```
%1 = insertelement
    <2 x i64> poison, i64 %in0, i32 0
%2 = insertelement
    <2 x i64> %1, i64 %in1, i32 1
%3 = add <2 x i64> %2, <i64 2, i64 5>
```

slp-vectorizer above. *load-store-vectorizer* collapses two **i64** loads at **%src**, **%src+1** into one **<2 x i64>** load; *loop-vectorize* turns **arg[i] = arg1[i] + arg2[i]** into **<8 x i16>** work per iteration.

Vectorizers Need a Real Target

```
target triple = "aarch64-apple-ios" ; <-- not decoration
```

Which vector types exist at all is target information, carried by `TargetLowering` under `TargetTransformInfo`. Drop the triple from a vectorizer test and you silently get a **default** target model:

*the pass still runs, still succeeds, and produces IR that
does not match what your compiler produces*

That is quiz 5. It is also the most likely way this chapter's material will actually bite you, because there is no error message – session 2's “IR is not fully target-independent”, with teeth.

`ch8/slp-vectorizer.ll` has its triple line **commented out**; `load-store-vectorizer.ll` and `loop-reduce.ll` do not. Uncommenting it is a one-line experiment.

The Recurring Theme

Count the “Target-specific” rows we just read:

API	Consulted by
TargetTransformInfo	argpromotion, inline, indvars, licm, loop-reduce, loop-unroll, simplifycfg, and all three vectorizers
TargetLibraryInfo	dce, indvars, licm, slp-vectorizer, loop-vectorize
TargetLowering	loop-reduce, load-store-vectorizer, loop-vectorize

Only `deadargelim` and `reassociate` consult nothing.

These are not trivia. They are the sockets.

Chapter 9 is: subclass `TargetTransformInfo`, supply your target’s costs, add your own intrinsics, and inject your passes into the default pipeline. Every generic pass above then optimizes **for your target** – without you writing any of them.

7. Hands-On

The Workflow: Read the Diff

Chapter 8's exercise is unlike the others – **no C++ to write**. Each pass has an input IR, and the exercise is to read what changed.

```
cd LLVM-Code-Generation/ch8
cmake -GNinja -DLLVM_DIR=$(llvm-config --cmakedir) -Bbuild .
ninja -Cbuild # runs every pass -> build/
xxx.out.ll
diff -U10 dce.ll build/dce.out.ll # the actual exercise
ninja -Cbuild -v # see the opt command used
touch dce.ll && ninja -Cbuild # re-run just one
```

19 inputs. Naming is `<cli_name>.ll`, with three exceptions – `xor.ll`, `canonical_form.ll`, and `value_tracking.ll` all exercise `instcombine`.

*Everything on the preceding slides was regenerated this way on LLVM 22.1.8 and matches the book (written against LLVM 20.1.1) **exactly**. This material is stable across versions – which is the book's own claim about itself.*

What This Module Adds

Upstream `ch8/` covers the optimization passes but has no input for the **first half** of the chapter. `08_passes/lab/` fills the gaps:

Stage	Gap it fills
<code>verify</code>	a non-dominating use, so the verifier actually fires
<code>mem2reg</code>	the one canonicalization pass with no upstream input
<code>analysis</code>	<code>print<loops></code> / <code>print<domtree></code> – analyses you can see
<code>printers</code>	nested <code>function(print, loop(print))</code> scoping in action
<code>scope-trap</code>	the two pipeline strings that fail, and why
<code>lcssa-ephemeral</code>	<code>lcssa</code> then <code>instcombine</code> : the phi disappears
<code>pass-order</code>	<code>indvars</code> then <code>simplifycfg</code> (quiz 4), correctly scoped

```
cd 08_passes/lab && ./run.sh          # all stages, with diffs
./run.sh mem2reg                       # just one
```

No CMake and no LLVM_DIR: nothing here compiles, it only runs `opt`. `./run.sh --check` additionally verifies each stage against its expected output.

Exercises

1. **exercises/01_find_the_unknown/** – the methodology, as a drill. You are given only a **description** (“removes redundant computations of the same expression”); find the CLI name, the scope, the source file, and a test that exercises it. Answer key lists the exact commands.
2. **exercises/02_pass_order/** – one input, several pipelines, very different outputs. Why does **mem2reg,dce** beat **dce,mem2reg**? Why does **instcombine** undo **lcssa**? What order would **you** put them in?
3. **exercises/03_write_an_id_card/** – pick any pass the chapter did **not** cover (**adce**, **gvn**, **sroa**, **jump-threading**, **loop-rotate**, ...), and produce its identity card: class name, CLI name, description, minimal In/Out IR you wrote yourself, and its target-specific dependencies.

Exercise 3 is the one that matters. If you can produce a correct card for a pass nobody explained to you, you have the skill this chapter is actually about – and you have extended the catalog by one entry.

Check Your Understanding (1/2)

- **Given a CLI name, how do you check it is part of the middle end?** Look for it in `opt --print-passes` under `Module`, `CGSCC`, `Function`, or `Loop`. If it only shows up in `--help`, or only under a `Machine ...` section, `opt` cannot run it as an IR pass.
- **An existing pass crashes on your custom pass's output. First thing to try?** Run the verifier on that output. Invalid IR must never reach a pass, so the bug is probably yours, not theirs.
- **Where is CSE implemented?** An optimization (`Transforms`), function-scoped (not `IPO`), non-vector (not `Vectorize`) → `llvm/lib/Transforms/Scalar/EarlyCSE.cpp`.
- **After `indvars`, what removes the leftover `br i1 false`?** `simplifycfg` – but `-passes='indvars,simplifycfg'` fails, because `indvars` sets a `loop` scope. Write `-passes='loop(indvars),simplifycfg'`.

Check Your Understanding (2/2)

- **Your load-store-vectorizer test does not produce the IR you see in your real compiler. Why?** Probably no `target triple` in the test, so `TargetLowering` supplied a default target model instead of your target's actual vector preferences.
- **You put the IR in LCSSA form early, and the pass that needs it sees no `.lcssa` values. What happened?** Something in between – `instcombine`, most likely – removed the extra phi. LCSSA is ephemeral; place it immediately before its consumer, or write a loop pass and get the guarantee for free.
- **Your target-specific rewrite keeps getting reverted. Why, and what do you do?** `instcombine` restored the canonical form. Stop running it after target-specific constructs appear, and use the `simplifyXXXInst` helpers instead if you still need the folds.
- **Why is the `-00` IR not canonical, and why does that matter?** `instcombine` is an optimization and does not run at `-00`. It matters because lowering passes run at **every** level, so they must be correct on non-canonical IR.